

Surprise Quiz–3 : Solutions

1. A comb polygon (the worst-case polygon for the Art Gallery problem) with exactly k prongs has $n = 3k$ vertices. For $k = 9$:
 - (i) How many triangles does any triangulation of this polygon contain? _____
 - (ii) What is the minimum number of cameras needed (worst case) to guard it? _____
 - (iii) How many diagonals are in the triangulation? _____

Solution: (i) 25, (ii) 9, (iii) 24

Explanation:

- $n = 3k = 27$. Triangles $= n - 2 = 25$.
- Cameras $= \lfloor n/3 \rfloor = \lfloor 27/3 \rfloor = 9$. Each prong requires its own camera.
- A triangulation of an n -vertex simple polygon always has exactly $n - 3$ diagonals. For $n = 27$: $27 - 3 = 24$.

2. The first floor of the LHC is modeled as a **simple polygon with 56 vertices**. You have **17 cameras**.

Which statements **must be true**? _____

- A. The building can always be covered
- B. The building can never be covered
- C. There exists a shape that cannot be covered
- D. There exists a shape that can be covered

Solution: C, D

Explanation:

This problem follows from the *Art Gallery Theorem*. The theorem states that for any simple polygon with n vertices,

$$\left\lfloor \frac{n}{3} \right\rfloor$$

guards (or cameras) are always sufficient to cover the entire polygon, and in the worst case this many guards may be necessary.

For the given polygon,

$$n = 56$$

Thus the maximum number of guards required in the worst case is

$$\left\lfloor \frac{56}{3} \right\rfloor = 18.$$

However, only 17 cameras are available.

Statement A

This would require that every polygon with 56 vertices can be covered using 17 cameras. Since there exist polygons that require 18 guards, this statement is **false**.

Statement B

This claims that no polygon with 56 vertices can be covered using 17 cameras. This is also false because some polygons (for example, convex polygons) can be covered with even fewer guards.

Statement C

Since the worst-case requirement is 18 cameras, there exists at least one polygon with 56 vertices that cannot be covered with 17 cameras. A classic example of such a worst-case polygon is the comb-shaped (or spiked) polygon used in proofs of the Art Gallery Theorem, where each spike hides a region from cameras placed in other spikes. In such constructions, roughly one guard is required for every three vertices.

An example of this construction is shown on page 27 of PDF slides 19, 20, 21, where a polygon with many narrow spikes requires $\lfloor n/3 \rfloor$ guards.

Thus, **Statement C is correct**.

Statement D

There exist polygons with 56 vertices that can be covered with fewer than 17 cameras.

Examples

- **Convex polygon**

In a convex 56-vertex polygon every point is visible from any interior point. Hence a single camera placed anywhere inside can observe the entire polygon.

- **Circle-like polygon**

A 56-sided polygon approximating a circle is convex and has no indentations blocking visibility. A camera placed near the center can see the entire interior.

- **Star-shaped polygon**

A star-shaped polygon has a point (called the kernel) from which the entire polygon is visible. Placing a camera at this point covers the whole polygon.

- **Polygon with two large halls**

Consider a 56-vertex polygon formed by two large rooms connected by a corridor. One camera in each hall can observe the entire structure, so only 2 cameras are sufficient.

- **Almost convex polygon**

A polygon that is mostly convex with only a few small indentations may require only a few cameras (for example 2 or 3) placed near those indentations to achieve full coverage.

Thus, there clearly exist polygons with 56 vertices that can be covered with 17 cameras.

Therefore, **Statement D is correct.**

Conclusion

The correct statements are

C and D

3. Which of the following statements about the **sweep-line algorithm for partitioning a polygon into y-monotone pieces** are correct?_____

- A. When a split vertex v is processed, a diagonal is inserted from v to $helper(left(v))$.
- B. When a split vertex v is processed, $helper(right(v)) \leftarrow v$.
- C. When processing an end vertex v , $helper(right(v)) \leftarrow v$.
- D. Diagonals are added at split vertices to remove local maxima that violate y-monotonicity.

Solution: A, B, D

Explanation: The sweep-line algorithm for partitioning a simple polygon into y -monotone pieces processes vertices in decreasing order of their y -coordinates while maintaining a status structure T of edges intersecting the sweep line. Each edge e in T stores a value $helper(e)$.

High-Level Algorithm

MakeMonotone(P)

1. Insert all vertices into a priority queue Q ordered by decreasing y .
2. Initialize an empty status structure T .
3. while Q is not empty:
4. $v \leftarrow \text{ExtractMax}(Q)$
5. case type(v) of
6. start vertex: $\text{HandleStartVertex}(v)$
7. end vertex: $\text{HandleEndVertex}(v)$
8. split vertex: $\text{HandleSplitVertex}(v)$
9. merge vertex: $\text{HandleMergeVertex}(v)$
10. regular vertex: $\text{HandleRegularVertex}(v)$

Handling a Split Vertex

HandleSplitVertex(v)

1. Let e be the edge in T immediately left of v .
2. Insert diagonal $(v, helper(e))$.
3. $helper(e) \leftarrow v$
4. Insert edge $right(v)$ into T
5. $helper(right(v)) \leftarrow v$

Step 2 inserts a diagonal from v to $helper(left(v))$.

Step 5 sets the helper of the edge leaving v downward to v .

Handling an End Vertex

HandleEndVertex(v)

1. if helper(prev(v)) is a merge vertex then
2. Insert diagonal (v , helper(prev(v)))
3. Remove prev(v) from T

No operation assigns $\text{helper}(\text{right}(v)) \leftarrow v$ for end vertices.

Evaluation of the Statements

A. When processing a split vertex v , the algorithm inserts a diagonal from v to helper of the edge immediately to the left of v .

This corresponds directly to Step 2 of the split vertex handler.

Thus, **A is correct.**

B. After processing the split vertex, the edge leaving v downward is inserted into the status structure and its helper is set to v .

This corresponds to Step 5 of the split vertex handler.

Thus, **B is correct.**

C. For an end vertex, the algorithm only checks whether the helper of the previous edge is a merge vertex and may insert a diagonal. No assignment $\text{helper}(\text{right}(v)) \leftarrow v$ occurs.

Thus, **C is incorrect.**

D. Split vertices correspond to local maxima that violate y -monotonicity. The algorithm inserts diagonals from these vertices to appropriate helper vertices to remove these maxima and partition the polygon into y -monotone pieces.

Thus, **D is correct.**

Conclusion

The correct statements are

A, B, D

4. Consider the following three algorithms for computing the convex hull of a set P of n points in $\mathbb{R}^2$, where h denotes the number of vertices on the output convex hull ($3 \leq h \leq n$):

- **Algorithm A (Graham's Scan):** Sorts all n points by polar angle and processes them in a single pass using a stack.
- **Algorithm B (Jarvis' March / Gift Wrapping):** Repeatedly selects the point that makes the smallest counter-clockwise turn from the current edge; uses $O(n)$ work per hull vertex.
- **Algorithm C (Chan's Algorithm):** Combines Graham's Scan on small subgroups with Gift Wrapping across groups; uses the doubling trick $m = \min\{n, 2^{2^t}\}$ to guess h .

Select ALL statements that are correct: _____

- (A) Algorithm A runs in $\Theta(n \log n)$ time for *any* input, even when $h = O(1)$.
- (B) Algorithm B runs in $O(n \log n)$ time in the worst case.
- (C) Algorithm C achieves $O(n \log h)$ time, which is optimal for output-sensitive convex hull computation.
- (D) When $h = n$ (all points are on the hull), Algorithm A is asymptotically faster than Algorithm C.
- (E) For $h = O(\log n)$, Algorithm B is asymptotically faster than Algorithm A.

Solution: A, C

Explanation:

- **(A) Correct.** Graham's Scan sorts n points in $\Theta(n \log n)$ time regardless of h .
- **(B) Incorrect.** Jarvis' March runs in $O(n \cdot h)$ time; worst-case (when $h = n$) this is $O(n^2)$, not $O(n \log n)$.
- **(C) Correct.** Chan's Algorithm achieves $O(n \log h)$ which matches the $\Omega(n \log h)$ lower bound for comparison-based output-sensitive convex hull.
- **(D) Incorrect.** When $h = n$, Chan's Algorithm also runs in $O(n \log n)$, matching Graham's Scan asymptotically.
- **(E) Incorrect.** If $h = O(\log n)$, then Jarvis' March runs in $O(nh) = O(n \log n)$, which is not asymptotically faster than Graham's Scan. It would be asymptotically faster only if $h = o(\log n)$.

5. Based on the fundamental definitions of graph structures, select **all** properties that are true for a **Tree**:_____

- A. The graph must be connected.
- B. The graph must contain at least one cycle.
- C. Removing any edge from the tree will disconnect the graph.
- D. A tree with n vertices must have exactly n edges
- E. It is classified as a “sparse” graph.

Solution: A, C, E

Explanation:

A. The graph must be connected. (True)

By definition, a tree $G = (V, E)$ must be connected. If a graph is acyclic but disconnected, it is a *forest*. For any two vertices $u, v \in V$, there exists at least one path between them.

B. The graph must contain at least one cycle. (False)

Trees are strictly **acyclic**. The presence of a cycle violates the definition of a tree.

C. Removing any edge will disconnect the graph. (True)

In a tree, there is exactly one unique path between any two vertices. Therefore, every edge $e \in E$ is a *cut-edge* or *bridge*. Removing any e results in the number of connected components increasing from 1 to 2.

D. A tree with n vertices must have exactly n edges. (False)

For any tree with n vertices, the number of edges $|E|$ is given by:

$$|E| = n - 1$$

A graph with n vertices and n edges that is connected must contain exactly one cycle.

E. It is classified as a “sparse” graph. (True)

A graph is considered sparse if $|E| \ll |V|^2$. Since a tree has the minimum number of edges required to maintain connectivity ($n - 1$), it is the most basic example of a sparse graph.

6. The naive recursive triangulation runs in $O(n^2)$. The sweep-line algorithm achieves $O(n \log n)$. Where does the $O(\log n)$ factor originate? _____
- (a) Sorting vertices in Q and performing BST operations on the sweep-line status T during each vertex event, each costing $O(\log n)$.
 - (b) The 3-coloring DFS on the dual tree, which requires $O(\log n)$ per node.
 - (c) The monotone triangulation step, which needs $O(\log n)$ per vertex for visibility checks.
 - (d) Computing the helper function, which requires $O(\log n)$ binary searches on the polygon boundary.

Solution: A

Explanation: The bottleneck is `MakeMonotone`.

Each of the n event steps involves:

- (i) `deleteMax` from Q in $O(\log n)$, and
- (ii) `find/insert/delete` in the balanced BST T in $O(\log n)$.

Total: $O(n \log n)$.

The DFS 3-coloring (b) is $O(n)$.

Monotone triangulation (c) is $O(n)$.

Helper lookups (d) use T , already in (a).

7. Let $ED(s_1, s_2)$ denote the edit distance between strings s_1 and s_2 of lengths m and n , respectively. Assume unit cost for insertion, deletion, and replacement operations. Select all statements that are correct. _____

- (a) $ED(x, z) \leq ED(x, y) + ED(y, z)$
- (b) $ED(s_1, s_2) \leq \max(m, n)$
- (c) The time complexity of computing edit distance using DP is $O(m + n)$
- (d) $ED(\text{"abcdefg"}, \text{"azced"}) = 3$
- (e) $ED(\text{"abcdefg"}, \text{"azced"}) = 4$

Solution: a, b, e

Explanation:

Let $ED(s_1, s_2)$ denote the edit distance between two strings s_1 and s_2 of lengths m and n , respectively. The allowed operations are insertion, deletion, and replacement, each having unit cost.

(a) $ED(x, z) \leq ED(x, y) + ED(y, z)$. Let $ED(x, y) = k$. Then there exists a sequence of k edit operations that transforms string x into string y . Similarly, let $ED(y, z) = \ell$. Then there exists a sequence of ℓ edit operations that transforms y into z . By first applying the k edits that convert x to y , and then applying the ℓ edits that convert y to z , we obtain a valid sequence of $k + \ell$ edits that transforms x directly into z . Since the edit distance $ED(x, z)$ is defined as the minimum number of edit operations required to transform x into z , it follows that

$$ED(x, z) \leq k + \ell = ED(x, y) + ED(y, z).$$

Thus, edit distance satisfies the triangle inequality, and statement (a) is **true**.

(b) $ED(s_1, s_2) \leq \max(m, n)$ We can always transform s_1 to s_2 by the following procedure:

- Replace the first $\min(m, n)$ characters of s_1 to match s_2 .
- Insert or delete the remaining $|m - n|$ characters.

The total number of operations required is at most

$$\min(m, n) + |m - n| = \max(m, n).$$

Hence

$$ED(s_1, s_2) \leq \max(m, n).$$

Thus statement (b) is **true**.

(c) Time complexity of DP algorithm The dynamic programming solution for edit distance fills a table of size $(m + 1) \times (n + 1)$. Each entry is computed in constant time. Therefore the time complexity is $O(mn)$ and not $O(m + n)$. Hence statement (c) is **false**.

(d) and (e) Compute $ED(\text{"abcdefg"}, \text{"azced"})$. One optimal sequence of operations is:

- Replace b with z
- Delete d
- Delete f
- Replace g with d

Thus the total cost is

$$ED(\text{"abcdefg"}, \text{"azced"}) = 4.$$

Hence statement (d) is **false** and statement (e) is **true**.

Final Answer:

(a), (b), and (e) are correct

8. Consider a directed graph $G = (V, E)$ where we perform a Breadth-First Search (BFS) starting from a specific vertex s . Let L_i denote the set of vertices discovered at Layer i . If there exists a directed edge $(u, v) \in E$ such that $u \in L_i$, which of the following statements is **always** true regarding the layer index of v , denoted as $L(v)$?_____

The i -th layer, denoted as L_i , is defined as:

$$L_i = \{v \in V \mid \text{dist}(s, v) = i\}$$

Where:

- $L_0 = \{s\}$
- $\text{dist}(s, v)$ is the length of the shortest path from s to v .
- For $i > 0$, a vertex $v \in L_i$ if it is adjacent to at least one vertex $u \in L_{i-1}$ and $v \notin \bigcup_{j=0}^{i-1} L_j$.

- (A) $L(v) = i + 1$
 (B) $L(v) \leq i + 1$
 (C) $L(v) \geq i$
 (D) $L(v) = i - 1$

Solution: B

Explanation:

In a Breadth-First Search (BFS), the layer index $L(u)$ of a vertex u is defined as the shortest path distance from the source s to u . Let $\text{dist}(s, u)$ denote this shortest path distance.

- (a) Given $u \in L_i$, we have by definition:

$$\text{dist}(s, u) = i$$

- (b) If there exists a directed edge $(u, v) \in E$, then there is a path from s to v passing through u with length $\text{dist}(s, u) + 1$.
 (c) By the properties of shortest paths, the actual shortest path distance from s to v must be less than or equal to the length of *any* specific path (including the one through u):

$$\text{dist}(s, v) \leq \text{dist}(s, u) + 1$$

- (d) Substituting the known value $\text{dist}(s, u) = i$:

$$L(v) \leq i + 1$$

Why Other Options are Incorrect

In a **directed** graph, an edge (u, v) where $u \in L_i$ can point to any previously discovered vertex or a new vertex. This results in three possible scenarios for $L(v)$:

- **Forward Edge:** If v is discovered for the first time via u , then $L(v) = i + 1$.

- **Cross Edge:** If v was already reached at the same level via a different path, then $L(v) = i$.
- **Backward Edge:** If v was already reached at an earlier level (e.g., v is the source s or in L_{i-k}), then $L(v) < i$.

Since $L(v)$ can be $i+1$, i , or any value less than i , the only statement that is **always** true is $L(v) \leq i + 1$.

9. You are given a graph representing a local social network with 100 individuals ($n = 100$). The graph is "Complete," meaning every person is directly connected to every other person. You perform a BFS starting from "Person A.". Determine the maximum layer index i reached upon the termination of the BFS, assuming the starting node is at Layer 0. _____

Solution: 1

Explanation:

In a complete graph K_{100} where every vertex is connected to every other vertex, the maximum layer index i reached upon the termination of the BFS starting from Person A is:

$$i_{max} = 1$$

Mathematical Explanation

The layer index L_i in a BFS corresponds to the shortest path distance from the starting vertex s . Let V be the set of individuals (vertices) such that $|V| = 100$.

- (a) **Layer 0 (L_0):** By definition, the starting node is at Layer 0.

$$L_0 = \{\text{Person A}\}$$

- (b) **Definition of a Complete Graph:** In a complete graph $G = (V, E)$, for every pair of distinct vertices $u, v \in V$, there exists an edge $(u, v) \in E$. This implies:

$$\forall v \in V \setminus \{s\}, \text{dist}(s, v) = 1$$

- (c) **Layer 1 (L_1):** BFS assigns a vertex v to layer L_i if the shortest path from the source s to v is exactly i . Since every person in the social network is directly connected to Person A:

$$L_1 = V \setminus \{\text{Person A}\}$$

This set contains the remaining 99 individuals.

- (d) **Termination:** After exploring L_1 , all vertices in the graph have been discovered ($L_0 \cup L_1 = V$). There are no remaining vertices to be placed into a Layer 2 or beyond.

Conclusion Regardless of the number of individuals n (as long as $n > 1$), the maximum layer index for a complete graph is always 1. The diameter of a complete graph is 1, meaning no two nodes are more than one step apart.

10. Let $G = (V, E)$ be a weighted undirected graph and let T be a **Minimum Spanning Tree (MST)** of G , maintained using adjacency lists. Suppose a new weighted edge $(u, v) \in V \times V$ is added to G .

The worst case time complexity of determining if T is still an MST of the resultant graph is: _____

- A. $\Theta(|E| + |V|)$
- B. $\Theta(|E||V|)$
- C. $\Theta(|E| \log |V|)$
- D. $\Theta(|V|)$

Solution: D

Explanation:

Let $G = (V, E)$ be a weighted undirected graph and let T be a Minimum Spanning Tree (MST) of G . Since T is a tree, it contains exactly $|V| - 1$ edges.

When a new edge (u, v) with weight $w(u, v)$ is added to G , inserting this edge into T creates exactly one cycle. This cycle consists of the new edge (u, v) and the unique path between u and v in the tree T .

By the **cycle property of MST**, in any cycle the edge with the maximum weight cannot belong to a Minimum Spanning Tree.

To determine whether T is still an MST:

- (a) Find the path between u and v in the tree T using DFS or BFS.
- (b) Identify the edge with the maximum weight on this path.
- (c) Compare this weight with $w(u, v)$.

Case 1: If

$$w(u, v) > w_{\max},$$

then the new edge is heavier than every edge in the cycle and therefore T remains an MST.

Case 2: If

$$w(u, v) < w_{\max},$$

then the maximum weight edge in the cycle must be removed and replaced with (u, v) to obtain the new MST.

Since T is a tree with $|V| - 1$ edges, finding the path between u and v using DFS/BFS takes

$$O(|V|).$$

Therefore, the worst-case time complexity of determining whether T is still an MST is

$$\boxed{\Theta(|V|)}.$$

11. Let $G = (V, E)$ be a simple undirected graph, and s be a particular vertex in it called the source. For $x \in V$, let $d(x)$ denote the shortest distance in G from s to x . A Breadth-First Search (BFS) is performed starting at s . Let T be the resultant BFS tree. If (u, v) is an edge of G that is not in T , then which one of the following **cannot** be the value of $d(u) - d(v)$?_____

- A. -1
- B. 0
- C. 1
- D. 2

Solution: D

Explanation:

Let $d(x)$ denote the shortest distance from the source vertex s to a vertex x in the graph G . A Breadth-First Search (BFS) starting from s explores vertices level by level according to their distance from s .

If u is at level i , then any neighbor v of u must lie in one of the following levels:

$$i - 1, \quad i, \quad \text{or} \quad i + 1.$$

Thus, for any edge (u, v) in G ,

$$|d(u) - d(v)| \leq 1.$$

Now consider the possible values:

- $d(u) - d(v) = -1$: possible if v is one level deeper than u .
- $d(u) - d(v) = 0$: possible if both vertices lie in the same BFS layer.
- $d(u) - d(v) = 1$: possible if u is one level deeper than v .

However, if $d(u) - d(v) = 2$, then $d(u) = d(v) + 2$. Since (u, v) is an edge, BFS would have discovered u through v with distance at most $d(v) + 1$, which contradicts $d(u) = d(v) + 2$.

Therefore, the value 2 cannot occur.

Answer: 2

12. In a **depth-first traversal** of a graph G with n vertices, k edges are marked as **tree edges** (tree edges are those edges which appear in the final DFS forest). The number of connected components in G is: _____

- A. k
- B. $k + 1$
- C. $n - k - 1$
- D. $n - k$

Solution: D

Explanation:

Tree edges are those edges which appear in the final DFS forest. For example, in the case of a connected graph (i.e. the DFS forest contains only one tree), the edges belonging to the DFS tree are called tree edges.

Assume that the graph has x connected components. Let the number of tree edges in the i -th component be K_i for $i = 1, 2, \dots, x$.

Thus, the total number of tree edges is

$$K_1 + K_2 + K_3 + \dots + K_x = K$$

Each component of the DFS forest is a tree. A tree with K_i edges contains $K_i + 1$ vertices. Therefore, the total number of vertices in the graph is

$$(K_1 + 1) + (K_2 + 1) + (K_3 + 1) + \dots + (K_x + 1) = n$$

Expanding the above expression gives

$$K_1 + K_2 + K_3 + \dots + K_x + x = n$$

Since

$$K_1 + K_2 + K_3 + \dots + K_x = K$$

we obtain

$$K + x = n$$

Hence, the number of connected components is

$$x = n - K$$

Number of connected components = $n - K$
--

13. Let T be a depth-first search tree in an undirected graph G . Vertices u and v are leaves of this tree T . The degrees of both u and v in G are at least 2. Which one of the following statements is true? _____

- A. There must exist a vertex w adjacent to both u and v in G .
- B. There must exist a vertex w whose removal disconnects u and v in G .
- C. There must exist a cycle in G containing u and v .
- D. There must exist a cycle in G containing u and all its neighbors in G .

Solution: D

Explanation: Let T be a depth-first search (DFS) tree of an undirected graph G .

Suppose u and v are leaves of the DFS tree T , and the degrees of both u and v in G are at least 2.

In a DFS tree of an undirected graph, every edge of G is either

- a **tree edge**, or
- a **back edge** connecting a vertex to one of its ancestors in the DFS tree.

Since u is a leaf of the DFS tree, it has no children in T . However, $\deg_G(u) \geq 2$, so apart from its parent in the DFS tree, u must be adjacent to at least one more vertex in G .

This additional edge cannot go to a new vertex (otherwise DFS would have visited that vertex from u , making it a child of u). Therefore, this edge must be a **back edge** from u to one of its ancestors in the DFS tree. Let this ancestor be w . Then the following edges exist:

$$u \rightarrow w \quad (\text{back edge})$$

and in the DFS tree there is a path

$$w \rightarrow \cdots \rightarrow u.$$

Combining the back edge with the tree path forms a cycle containing u .

Now consider all neighbors of u in G . Each such neighbor must be either

- the parent of u , or
- an ancestor of u (via a back edge).

All these vertices lie on the DFS tree path from the root to u . By using the back edges from u to its ancestors together with the tree edges along this path, we obtain a cycle that includes u and all its neighbors. Thus, there exists a cycle in G containing u and all its neighbors.

Option D is correct.

14. Let $G = (V, E)$ be an undirected unweighted connected graph. The diameter of G is defined as

$$\text{diam}(G) = \max_{u, v \in V} \{\text{length of the shortest path between } u \text{ and } v\}$$

Let M be the adjacency matrix of G . Define a graph G_2 on the same vertex set with adjacency matrix N , where

$$N_{ij} = \begin{cases} 1 & \text{if } M_{ij} > 0 \text{ or } P_{ij} > 0, \text{ where } P = M^2 \\ 0 & \text{otherwise} \end{cases}$$

Which one of the following statements is true? _____

- A. $\text{diam}(G_2) \leq \lceil \text{diam}(G)/2 \rceil$
- B. $\lceil \text{diam}(G)/2 \rceil < \text{diam}(G_2) < \text{diam}(G)$
- C. $\text{diam}(G_2) = \text{diam}(G)$
- D. $\text{diam}(G) < \text{diam}(G_2) \leq 2 \text{diam}(G)$

Solution: A

Explanation: Let $G = (V, E)$ be an undirected, unweighted, connected graph with adjacency matrix M . The diameter of G is defined as

$$\text{diam}(G) = \max_{u, v \in V} \{d_G(u, v)\}$$

where $d_G(u, v)$ is the length of the shortest path between vertices u and v in G .

A new graph G_2 is defined on the same vertex set with adjacency matrix N , where

$$N_{ij} = \begin{cases} 1 & \text{if } M_{ij} > 0 \text{ or } P_{ij} > 0, \text{ where } P = M^2 \\ 0 & \text{otherwise} \end{cases}$$

Interpretation of M^2 : For an adjacency matrix M , the entry $(M^2)_{ij}$ gives the number of walks of length 2 between vertices i and j . Hence $P_{ij} > 0$ implies that there exists a path of length 2 between i and j in G . Thus, in the graph G_2 , two vertices are adjacent if they are connected by a path of length 1 or 2 in G .

Relation between distances in G and G_2 : Let the shortest path between vertices u and v in G have length

$$d_G(u, v) = k.$$

In G_2 , a single edge can correspond to either

- a path of length 1 in G , or
- a path of length 2 in G .

Therefore, a path of length k in G can be traversed in at most

$$\left\lceil \frac{k}{2} \right\rceil$$

steps in G_2 . Hence

$$d_{G_2}(u, v) \leq \left\lceil \frac{d_G(u, v)}{2} \right\rceil.$$

Diameter of G_2 : Taking the maximum over all pairs of vertices,

$$\text{diam}(G_2) = \max_{u, v \in V} d_{G_2}(u, v) \leq \max_{u, v \in V} \left\lceil \frac{d_G(u, v)}{2} \right\rceil = \left\lceil \frac{\text{diam}(G)}{2} \right\rceil.$$

Thus,

$$\boxed{\text{diam}(G_2) \leq \left\lceil \frac{\text{diam}(G)}{2} \right\rceil}$$

Hence, the correct option is **A**.

15. Let $U = \{1, 2, 3\}$. Let 2^U denote the powerset of U . Consider an undirected graph G whose vertex set is 2^U . For any $A, B \in 2^U$, (A, B) is an edge in G if and only if (i) $A \neq B$, and (ii) either $A \subsetneq B$ or $B \subsetneq A$.

For any vertex A in G , the set of all possible orderings in which the vertices of G can be visited in a Breadth First Search (BFS) starting from A is denoted by $B(A)$.

If $\emptyset$ denotes the empty set, then the cardinality of $B(\emptyset)$ is: _____

Solution: 5040

Explanation: Let $U = \{1, 2, 3\}$. Then the power set of U is

$$2^U = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

Thus the graph G has 8 vertices. Two vertices $A, B \in 2^U$ are connected by an edge if

$$A \neq B \quad \text{and} \quad (A \subset B \text{ or } B \subset A)$$

that is, one set is a proper subset of the other. We perform Breadth First Search (BFS) starting from the vertex $\emptyset$.

Level 0: $\emptyset$

Level 1: All sets that contain $\emptyset$ as a proper subset:

$$\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}$$

Thus there are 7 neighbors of $\emptyset$. During BFS, all these 7 vertices are discovered immediately when processing $\emptyset$. Therefore, the remaining vertices can appear in *any order* in the BFS traversal after $\emptyset$. Hence the number of possible BFS orderings is the number of permutations of these 7 vertices:

$$|B(\emptyset)| = 7! = 5040$$

$$\boxed{5040}$$